Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Metric-based analysis to identify anomalous releases in the NPM ecosystem Software Supply Chain

Michele Frattini

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/

Computer Science MSc
Software Security and Engineering Curriculum

Metric-based analysis to identify anomalous releases in the NPM ecosystem Software Supply Chain

Michele Frattini

Advisor: Giovanni Lagorio

Examiner: Luca Caviglione

March, 2026

Table of Contents

Chapter 1

Introduction

Chapter 2

Background

In this chapter, I provide the fundamental information regarding the Software Supply Chain (SSC), Software Supply Chain Attack (SSCA), the Node Package Manager (NPM) ecosystem and the tarballs.

2.1 Software Supply Chain and NPM Ecosystem

Developers tend to use ready-made software components in their projects for common functionalities (such as date parsing, HTTP request handling, or User Interface (UI)), rather than implementing them from scratch.

These software components are called dependencies, and these dependencies in turn can depend on other components, thus constituting the SSC [OS23].

These components or packages are distributed in Open Source Package Repositories such as NPM, Python Package Index (PyPI) and RubyGems.

These repositories are often used to carry out attacks on the SSC, as they are open and decentralized systems where anyone can freely publish a package [Ohm+20]. An attacker may inject malicious code into a legitimate package update, in order to compromise in this way all projects and packages that depend on it.

Several attack vectors are possible [Dua+20], such as Account Compromise, which refers to compromising accounts of Package Maintainers (those who develop and manage the packages) on the registry portal (like NPM), allowing the attacker to release malicious versions.

In SSCA the victims are developers and users. Developers can be exploited directly to steal

their credentials or damage their infrastructure, and indirectly as a channel to reach users through their applications or services. Users can be exploited to steal their credentials or damage their devices.

Attackers often attempt to mask their malicious code using obfuscation techniques to prevent visual detection [Ohm+20]. The most common technique is to use a different encoding, such as hexadecimal, to hide function or variable names, renaming them with names devoid of semantic meaning like _0x264994. The leading underscore is inserted because in JavaScript, and in other programming languages, variable or function names cannot start with a number. In the malicious code, however, there are also hexadecimal values without this prefix, used for example to simply represent numbers and constants. This approach is simple and effective, as it does not require the use of external dependencies.

All the information regarding NPM reported below has been taken from the official documentation [npm26].

For the JavaScript and Node.js ecosystem, NPM is one of the largest and most active Open source package repositories, with over two million packages available [Moo+21].

The public NPM registry is a database of JavaScript packages, where developers can share and download packages.

A package is a folder containing various files that can be borrowed for use in own projects.

To resolve packages by name and version, NPM communicates with the public registry (configured by default at https://registry.npmjs.org), from which allows users to retrieve any version of a package currently available in the registry.

A NPM tarball is a gzipped package, the standard format used when packages are uploaded to a registry, and represents a package at a specific version. Instead, a Git URL refers to a package in a Git repository.

The content of the tarball may differ from the content of the Git repository, even though both represent the same package.

In fact, through the optional "files" field of package.json, it is possible to include only the specified files, or exclude them if the .npmignore file is present.

Therefore, to ensure a fair and symmetrical comparison, it is necessary to analyze the same type of artifact.

2.2 Tarball content

A tarball can contain any type of file, however, the following types of files are typically found within it [Pin+23].

- **Configuration files and Metadata:**

  The package.json file describes the package's file or directory, and a package must contain a package.json file in order to be published to the NPM registry.

  In package.json, the name and version fields are required, and together they form an identifier that is assumed to be completely unique.

  Once a package is published with a given name and version, that specific combination can never be used again, even if it is removed with `npm unpublish`. This is the reason why it is impossible to overwrite malicious versions once they have been released.

  Other common optional fields include the description, license, repository and dependencies on other packages. It is important to note that specifying a reference (a link) to a GitHub repository for a package is optional.

  Duan et al. [Dua+20] highlight that this lack of mandatory link represents a significant security gap because it prevents registries from verifying that the distributed package actually matches its public source code, allowing attackers to inject malicious code during the publication phase without detection.

- **Documentation and general information:**

  Frequently, documentation files, such as Markdown files like README.md and CHANGELOG.md, and the LICENSE text file used to specify copyright, are included inside the tarball.

- **Code files and Build Artifacts:** The tarball generally includes JavaScript and/or TypeScript code files, which constitute the functional logic of the package.

  Build artifacts are often found within the tarball [Gos+20]. These files are the final product of the transformation and compilation process of a package's source code. They are optimized versions of the original code automatically generated, rather than written directly by developers.

  Through processes such as minification, these artifacts reduce the size of the files and improving application performance.

  Furthermore, this also ensures compatibility, as the code transformation during the build allows the software to run across different environments and runtimes without the end user needing to worry about the original syntax.

Since the NPM ecosystem is permissive, there is no standardized way to distinguish if a package contains build artifacts or where they are located. While common conventions suggest that folders such as `dist/`, `build/` or `lib/` contain artifacts, this structure is neither guaranteed nor verifiable a priori, as it depends entirely on the developer's choices.

Chapter 3

Related Work

To counter SSCA, several detection tools have been developed to detect malicious code in NPM packages.

DONAPI [Hua+24a] is one of these tools, which uses a combination of static analysis, based on a predefined database of sensitive Application Programming Interfaces (APIs), and dynamic analysis, to extract behavioral patterns in order to detect and classify malicious packages.

Another tool is SpiderScan [Hua+24b], which is based on graph-based behavior modeling and matching. It uses an Large Language Model (LLM) for various tasks, including the identification of sensitive APIs and the analysis of shell commands. SpiderScan is not the only tool that uses LLMs, as many others exist [Zah+25; Wan+25; Zha+25].

These two tools are not open source and have some limitations.

For example the dynamic analysis in DONAPI risks hindering its practical usefulness in real-world scenarios due to its heavyweight nature and increased slowness [Hua+24b].

In SpiderScan, instead, the use of prompt engineering methods that leverage the capability of LLM to detect maliciousness can be highly expensive to use in real-world scenarios.

Furthermore, these tools analyze only the last published release of the packages, without considering the anlysis of more release to obtain a vision of the evolution of the package over time.

A tool that instead analyzes the evolution of the package is the one by Zoratti [Zor25]. His research is focused on the identification of specific metrics, such as the whitespace to printable character ratio (W/P ratio), to identify Blank Space Trojan (BST) and Hidden Unicode Trojan (HUT) attacks in GitHub repositories.

Chapter 4

Analysis of Recent Attacks

In this chapter, we will present the four relevant SSCA that occurred in 2025 in the NPM ecosystem, which have been selected as case studies for this thesis.

Each of these attacks will be analyzed in detail, how it works and what its main characteristics are, and introduced, absent in its benign versions.

The four attacks will be presented in chronological order, from the oldest to the most recent.

4.1 Attack A: Malicious Windows DLL

The first attack, which occurred on July 18, 2025, infected five NPM packages, one of which, eslint-config-prettier, is widely used with over thirty million weekly downloads [Lab25].

A maintainer of this package fell victim to a phishing email, and the attacker, once the account was compromised, published malicious versions of the following packages.

- eslint-config-prettier
- eslint-plugin-prettier
- synckit
- @pkgr/core
- napi-postinstall

Inside the tarball of these malicious versions, the attackers added a malicious Dynamic Link Library (DLL) weighing 1.2 MB.

Through a script also present within the tarball, it executed the malicious DLL and gave attackers remote-code execution on Windows machines, to, for example, install additional malware or exfiltrate tokens.

4.2 Attack B: Crypto attack

On September 8, 2025, an attacker compromised eighteen popular NPM packages, including chalk and debug, respectively the first and third most popular packages (in terms of number of stars) on NPM [F-R26].

These eighteen packages collectively see over 2.6 billion downloads each week, making the attack one of the most impactful SSCAs in the recent history of the NPM ecosystem [HH25].

Although the attack in terms of financial losses it was limited, as the compromised versions of the packages remained online for a short time (about two hours) before being detected by developers and removed from NPM.

The attack was made possible after compromising the account of a maintainer (qix) of one of the packages through a phishing email.

After obtaining the account credentials, the attacker published malicious versions of these packages, which allowed them to steal cryptocurrencies by intercepting and manipulating users' transactions.

The malware's operation can be divided into various phases [Mat25].

First, it was checked for the presence of a wallet in the browser, such as MetaMask.

Subsequently, if it is present, the malicious code waits for the user to send cryptocurrencies, and in this case, hooked the functions used to manage cryptocurrency transactions (signing and sending transactions), such as in the Ethereum ecosystem, the function ethsendTransaction, while in Solana, the function signAndSendTransaction.

Once one of these functions was hooked, the malicious code replaced the transaction's destination address with one controlled by the attacker, thus redirecting the cryptocurrencies toward a wallet owned by the attacker.

The malicious code was all inserted into the index.js files as a single obfuscated line of code 76,438 characters long, which also increased the file weight.

The obfuscation technique used here is to encode in hexadecimal function names, variables,

and constants, making the index.js file full of hexadecimal values such as _0x6fd57a or 0x1c8, compared to the index.js file of benign versions.

The presence of a wallet in the browser, the hooking of functions and the replacement of the transaction's destination address are behaviors made hidden by obfuscation, and therefore cannot be detected through static analysis.

Instead, the actual addresses of the attacker's wallets are visible in the code, and there are seven of them.

4.3 Attack C: Shai-Hulud 1.0

On September 15, 2025, 187 NPM packages, including @ctrl/tinycolor with over two million weekly downloads, were infected with malicious code designed to steal sensitive information such as tokens and authentication keys [BC25].

The method of the first infection and the patient zero remained unknown, but it could be a case of phishing [Tea25a]. The number of infected packages is so high because the malware automatically publishes a malicious version of every package of the compromised maintainer (of which it has publishing permissions).

The malicious code was inserted into the packages' tarballs in the new file bundle.js.

The file weighs approximately 3.7 MB and contains only a single line of code 3,734,355 characters long.

When the victim finishes installing the package with the infected version, the malicious script is executed and downloads a platform-appropriate version of TruffleHog, a legitimate tool for searching secrets such as cryptographic keys and API tokens in local file systems and Git repositories.

After starting and completing the search with TruffleHog, the malicious code publishes the stolen data by creating a public repository named Shai-Hulud (like the worm in the Dune novels) on behalf of the victim using their GitHub token.

In this repository, there is a file data.json containing all the collected secrets and system information encoded.

4.4 Attack D: Shai-Hulud 2.0

Only two months after the Shai-Hulud 1.0 attack Section 4.3, on November 24, 2025, a second, more aggressive attack compared to the previous one occurred, renamed Shai-Hulud 2.0, which this time infected 1,055 NPM packages within hours [Cut25].

The objective remained unchanged, namely to steal sensitive information such as tokens and authentication keys, and the self-propagation also remained the same.

However, this new attack shows differences compared to its previous version Section 4.3 [Tea25b].

The new versions of these packages, published in the NPM registry, claimed to introduce the Node.js alternative Bun runtime, inserting into the tarball a new installation file setup_bun.js and a file bun_environment.js.

The first file mimics the code related to the Bun configuration and calls the second file bun_environment.js.

The second file, weighing over 10 MB, contains a single obfuscated line of code 10,157,373 characters long.

As in the previous version Section 4.3, once the script (bun_environment.js) is started, it downloads and executes a version of TruffleHog with the purpose of stealing sensitive information such as NPM tokens, Amazon Web Services (AWS), Google Cloud Platform (GCP), and Azure credentials, and environment variables.

After finishing the search, the malicious code publishes the collected secrets and system information encoded by creating a public repository on behalf of the victim named Sha1-Hulud: The Second Coming, confirming that it is the successor of the previous Shai-Hulud 1.0 attack Section 4.3.

In this new version of the attack, however, the malicious code makes use of an obfuscation technique that encodes in hexadecimal function names, variables, and constants, making the bun_environment.js file full of hexadecimal values.

Even though the code is obfuscated, words like "password" and "aws_secret" are visible and present in clear text, making it possible to identify them through static analysis.

Chapter 5

Experimental Setup

This chapter describes the datasets used for the analyses conducted in Chapter 6 and the software tools developed for their creation Section 5.2.

5.1 Datasets

The two datasets, Goodware dataset Section 5.1.1 and Malware dataset Section 5.1.2, are a collection of metrics, quantifiable code characteristics, extracted from a set of NPM packages, each with 20 versions.

The metrics are identical for both datasets, and to extract this data, a software tool was developed, which is described in Section 5.2.1. The rationale behind the selection of these metrics and their analysis is explained for each metric in its own section in Chapter 6.

The packages selected for the two datasets are different. However, for both datasets, 20 versions were collected for each package to study how they behave and evolve across versions.

The two datasets are comparable to each other as they are homogeneous, since both have packages with the same number of versions (20).

5.1.1 Goodware dataset

This dataset is composed of packages ranked among the 1,000 most popular in the NPM registry, in terms of number of stars [F-R26].

Out of these 1,000 packages, 371 have less than 20 versions available in the NPM registry.

Therefore, the dataset includes the remaining 629 packages, with their 20 most recent available versions.

5.1.2 Malware dataset

This dataset is composed of 20 NPM packages compromised by one of the four SSCA analyzed in Chapter 4.

For these 20 packages, the malicious versions related to their respective attack were retrieved, as explained in Section 5.2.2, along with the 19 most recent versions available in the NPM registry.

Consequently, this dataset is composed of 20 packages, each with 20 versions: one malicious and 19 legitimate.

The list below reports the compromised packages in the Malware dataset for each attack, including the specific malicious version number.

- For the attack A Section 4.1 there are 3 packages:
  - eslint-config-prettier, version 10.1.7
  - eslint-plugin-prettier, version 4.2.3
  - synckit, version 0.11.9
- For the attack B Section 4.2 there are 9 packages:
  - ansi-regex, version 6.2.1
  - ansi-styles, version 6.2.2
  - chalk, version 5.6.1
  - color-convert, version 3.1.1
  - color-string, version 2.1.1
  - debug, version 4.4.2
  - supports-color, version 10.2.1
  - strip-ansi, version 7.1.1
  - wrap-ansi, version 9.0.1
- For the attack C Section 4.3 there are 4 packages:
  - @ctrl/deluge, version 7.2.1
  - @ctrl/shared-torrent, version 6.3.2

  - @ctrl/tinycolor, version 4.1.2
  - ember-browser-services, version 5.0.3

- For the attack D Section 4.4 there are 4 packages:
  - @asyncapi/avro-schema-parser, version 3.0.26
  - @asyncapi/bundler, version 0.6.6
  - @asyncapi/cli, version 4.1.3
  - posthog-node, version 5.11.3

The Malware dataset only includes packages with at least 19 versions available in the NPM registry.

However in these attacks, there are compromised packages that have fewer than 19 versions. In such case, they were ignored and not included in the dataset. Specifically, two packages were excluded from attack A and nine from attack B.

Attacks C and D involved a large number of compromised packages, respectively 187 and 1,055. For this reason, it was decided to include in the dataset a limited number of the most relevant packages for both attacks, specifically those with a high number of weekly downloads (at least over 1,000).

The purpose of this dataset is to provide a set of metrics for packages compromised by real-world attacks. Therefore, it is not required for these packages to be among the 1,000 most popular in the NPM registry. In the list, all nine packages from attack B are among the 1,000 most popular, while for the other attacks, none of the compromised packages fall into that category.

5.2 Software Tools

In this section, the developed tool and script will be described, along with the approaches used, in order to create the two datasets described in Section 5.1.

5.2.1 Tool for Analyzing NPM Packages

In this subsection, the tool `Analyzer-npm-package-releases` is described, which was developed to extract the metrics of Chapter 6 from a set of NPM package versions.

Using this tool, it was possible to construct the Goodware dataset (Section 5.1.1) and the Malware dataset (Section 5.1.2).

Implemented in Python, it is available on GitHub[1], and can be invoked from the command line in the following way:

```
python3 main.py --json packages.json [--workers N] [--local]
```

The tool requires as input a JavaScript Object Notation (JSON) file containing a list of names of NPM packages to be analyzed.

The work is executed in a parallel manner using multiprocessing, and the `--workers` flag allows for specifying the number of processes to be used.

The `--local` flag instead allows for including local tarballs of NPM package versions in the analysis.

The tool processes one package and one version at a time in increasing chronological order, that is, from the oldest version to the most recent one. Instead, multiprocessing is applied at the level of the single package version, analyzing the files that compose it in parallel.

The work of the tool is structured into three phases:

- Version Retrieval
- Metrics Calculation
- CSV Reporter

Version Retrieval

First, the NPMClient component queries the official NPM registry to retrieve the package metadata in order to obtain its list of available versions.

From this list, only versions parsable via node-semver [Pre] are kept, to make them chronologically sortable.

For the Goodware dataset (Section 5.1.1), the tool selects the 20 most recent valid versions from the list and downloads the related tarballs.

Instead for the Malware dataset (Section 5.1.2), the tool selects and downloads the 19 most recent valid tarballs versions.

The twentieth version is the tarball related to the malicious version, retrieved as explained in Section 5.2.2 and inserted via the `--local` flag.

[1] https://github.com/Frazzerz/Analyzer-npm-package-releases

Metrics Calculation

In this phase, the metrics are first calculated at the level of the single file of the tarball.

The calculation of each metric is different, and each one is described in depth in its own section in Chapter 6.

Metrics based on the textual content of the files, such as Unique Hexadecimal values (**??**), are calculated only on plain text files, such as JavaScript, TypeScript, JSON and Markdown files, ignoring binary files such as zip, png or dll.

Furthermore, for these metrics, comments are removed from JavaScript and TypeScript files using the parser from the tree-sitter library [Bru].

Subsequently, the metrics calculated on single files are aggregated to form metrics at the package version level.

For example, to calculate the package size metric (**??**), the weights of all files calculated individually first are summed.

Aggregation strategies vary based on the type of metric. For instance, counts and weights are summed, the maximum value among the longest lines of the files is taken, and strings are collected into lists.

CSV Reporter

Finally, for each package, the results are saved incrementally in two CSV files.

- `file_metrics.csv` contains one row for every file of each package version. The columns represent the metrics at the single file level.
- `aggregate_metrics_by_single_version.csv` contains one row for every package version, and the columns represent the metrics at the package version level.

Incremental saving is performed at the the end of the analysis of each version, and ensures that partial results are preserved even in case of error.

In addition to the CSV files, the tool also produces a log.txt log file, that is also updated incrementally.

Inside the file, there is general information such as the number of packages to analyze and the number of workers that will be used. Furthermore, for each package, the file also reports: the number of versions found in the registry, the status of the tarball downloads, the progress of the analysis of the selected versions and the total time for the analysis of the entire package.

5.2.2 Download Malicious Tarballs

In this subsection, the process and the developed script used to retrieve the tarballs of the malicious versions of the packages specified in Section 5.1.2 are described.

Public Repositories of Malicious Packages

The first approach used was to consult public datasets on GitHub, such as the one by Datadog [Dat23].

These public repositories, which contain malicious versions of packages from a registry, are active and regularly updated. Their limitation is that they do not always contain specific malicious versions of compromised packages.

In fact, in our case, the versions of the packages compromised by the attacks in Chapter 4 that we were looking for were not initially present in the dataset by Datadog [Dat23].

Mirror registry

The alternative used was to consult mirror registries, which are servers that replicate the content of official package repositories such as NPM, PyPI, and RubyGems [Guo+23].

Some examples of mirror registries include Huaweicloud, npmmirror, Tencent, and Alibaba.

The main characteristic of a mirror is its synchronization frequency with the official registry. Such synchronization varies depending on the mirror and can be more or less frequent. However, since synchronization does not occur instantaneously, mirrors can preserve versions of packages already removed from the official registry for extended periods.

This latency makes mirrors a useful tool for retrieving malicious versions of packages after their removal from official registries.

Script to Download Malicious Tarballs

To automate the search and download across different mirrors, the script `downloadpkgnpm.py` was developed.

Implemented in Python and available on GitHub[2], the script can be invoked from the command line in the following way:

[2] https://github.com/Frazzerz/Download-malicious-npm

```
python3 downloadpkgnpm.py json_file.json
```

The script requires as input a JSON file where one can specify a list of mirrors to query and a list of packages and versions to download.

For each package, the script sequentially queries the specified mirrors in search of the required version.

If the tarball is found, the script proceeds to download it. Otherwise, if the search fails on all mirrors, the operation's failure is notified.

In both cases, the execution continues automatically with the search for the next package until the list is exhausted.

With this script, it was possible to retrieve the malicious versions of the compromised packages used in the tool `Analyzer-npm-package-releases` (Section 5.2.1) to create the Malware dataset (Section 5.1.2).

Chapter 6

Metrics Analysis

In this chapter, we present the metrics of this study, the analysis of their behavior in the datasets (Section 5.1) and evaluate their effectiveness in detecting the attacks (Chapter 4).

There are countless metrics based on code analysis [Wei+25]. For example, in the study conducted by Weissberg et al. [Wei+25] to identify vulnerabilities in C and C++ code, they considered metrics related to:

- Code Smell such as the number of goto statements and number of function pointers
- Complexity metrics such as the number of loops and number of local variables
- Metrics on memory management operations
- Abstract Syntax Tree (AST) metrics

For this study, we defined and analyzed a series of metrics developed based on the most relevant characteristics of one or more SSCA (Chapter 4).

We discuss each metric in a dedicated subsection, where:

1. we define the metric and explain how it was calculated.
2. we analyze the behavior of the packages in the Goodware dataset (Section 5.1.1) for that metric, and examine the nature of packages that assume anomalous values.
3. we analyze the behavior of the packages in the Malware dataset (Section 5.1.2) for the metric, comparing it with the behavior of the Goodware dataset. Our goal is to verify the effectiveness of the metric in detecting attacks (Chapter 4).

In this study, we use figures with the same characteristics (unless otherwise specified) to represent the behavior of the metrics. Below, we explain how to interpret them.

Figure Behavior of the Goodware dataset for a Metric and Outlier Analysis

This figure illustrates the trend across 20 versions of the metric in the Goodware dataset (Section 5.1.1), after excluding the packages that are not significant in this metric.

The x-axis reports the package versions, from the least recent (-19) to the latest available (0), while the y-axis represents the number of occurrences of the metric in a logarithmic scale.

The heights of the vertical black bars represent the confidence intervals, specifically, the intervals containing 95% of the package population for a given version. Packages positioned within one of these intervals have a number of occurrences ranging between the minimum value and the 95th percentile of the population for that version.

Instead, the gray dots represent the outlier values, which are the remaining 5% of the population positioned outside the confidence interval.

A package assumes outlier values for a version when its number of values in that version exceeds the 95th percentile value of that confidence interval.

We have highlighted with colored lines the packages that have outlier values or values that are the upper limit of a confidence interval for one or more versions.

These packages, and their relative outlier values, exhibit atypical behavior compared to the majority of the population. For this reason, they have been analyzed individually to understand their nature and the reasons leading to such high values.

The highlighted packages may show an increasing, decreasing, or stable trend, and for better visualization, if there are many highlighted packages, we provide separate figures for each trend.

The red dashed line represents the median value calculated for each package version. Unlike the mean, the median is not influenced by outlier values and therefore does not distort the actual behavior of the metric.

The median and the y-axis logarithmic scale were used to highlight heterogeneous between packages in the dataset, as there are packages with a low number of values and packages with a number of values that exceed those of the majority of the population by several orders of magnitude.

Figure Comparison of Malware and Goodware Datasets Behavior for a Metric

The figure illustrates the trend across 20 versions of the metric in the Malware dataset (Section 5.1.2) compared with the Goodware dataset (Section 5.1.1).

The x-axis reports the package versions, from the least recent (-19) to the latest available (0), while the y-axis represents the number of occurrences of the metric in a logarithmic scale.

The figures show the packages of the Malware dataset for each of the four attacks with colored lines and their compromised malicious versions with red crosses.

Furthermore, the figures show the confidence intervals (vertical black bars) and the outlier values (gray dots) explained in Chapter 6, calculated on the Goodware dataset across the 20 versions.

If the packages of the Malware dataset overlap for a specific attack, we provide a separate figure for each package of that attack to ensure better visualization.

In these figures, one can see if the metric is sensitive enough to detect the compromised versions of the packages.

To make malicious versions identifiable, they must be placed among the outlier values.

If however, the malicious versions have a number of occurrences to fall within the confidence intervals, the metric will not be indicative for detecting that attack, since the versions would be indistinguishable from the legitimate ones.

Chapter 7

Conclusion

Bibliography

[Bru] Max Brunsfeld. *Tree-sitter*. URL: %5Curl%7Bhttps://tree-sitter.github.io/tree-sitter/%7D.

[BC25] Moshe Siman Tov Bustan and Alexander Chailytko. *180+ NPM Packages Hit in Major Supply Chain Attack*. 2025. URL: https://www.ox.security/blog/npm-2-0-hack-40-npm-packages-hit-in-major-supply-chain-attack/.

[Cut25] Gianpietro Cutolo. *Shai-Hulud 2.0: Aggressive, Automated, and Fast Spreading*. 2025. URL: %5Curl%7Bhttps://www.netskope.com/blog/shai-hulud-2-0-aggressive-automated-one-of-fastest-spreading-npm-supply-chain-attacks-ever-observed%7D.

[Dat23] Datadog. Mar. 2023. URL: https://github.com/datadog/malicious-software-packages-dataset.

[Dua+20] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. *Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages*. 2020. arXiv: 2002.01139 [cs.CR]. URL: https://arxiv.org/abs/2002.01139.

[F-R26] Tristan F.-R. *List of top 10000 NPM packages*. 2026. URL: https://tristan-f-r.github.io/npm-rank/PACKAGES.html.

[Gos+20] Pronnoy Goswami, Saksham Gupta, Zhiyuan Li, Na Meng, and Daphne Yao. "Investigating The Reproducibility of NPM Packages". In: *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2020, pp. 677–681. DOI: 10.1109/ICSME46990.2020.00071.

[Guo+23] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. *An Empirical Study of Malicious Code In PyPI Ecosystem*. 2023. arXiv: 2309.11021 [cs.SE]. URL: https://arxiv.org/abs/2309.11021.

[HH25] Asaf Henig and Cameron Hyde. *Breakdown: Widespread npm Supply Chain Attack Puts Billions of Weekly Downloads at Risk*. https://www.paloaltonetworks.com/blog/cloud-security/npm-supply-chain-attack/. 2025.

[Hua+24a] Cheng Huang, Nannan Wang, Ziyan Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. *DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping*. 2024. arXiv: 2403.08334 [cs.CR]. URL: https://arxiv.org/abs/2403.08334.

[Hua+24b] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. "SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching". In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ASE '24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 1146–1158. ISBN: 9798400712487. DOI: 10.1145/3691620.3695492. URL: https://doi.org/10.1145/3691620.3695492.

[Lab25] Endor Labs. *CVE-2025-54313: eslint-config-prettier Compromise — High Severity but Windows-Only*. 2025. URL: https://www.endorlabs.com/learn/cve-2025-54313-eslint-config-prettier-compromise----high-severity-but-windows-only.

[Mat25] Nicolas Matthee. *The npm Supply Chain Attack of September 2025*. 2025. URL: %5Curl%7Bhttps://www.prventi.com/blog/the-npm-supply-chain-attack-of-september-2025%7D.

[Moo+21] Marvin Moog, Markus Demmel, Michael Backes, and Aurore Fass. "Statically Detecting JavaScript Obfuscation and Minification Techniques in the Wild". In: *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2021, pp. 569–580. DOI: 10.1109/DSN48987.2021.00065.

[npm26] Inc. npm. *npm Documentation*. https://docs.npmjs.com/. 2026.

[Ohm+20] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. *Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks*. 2020. arXiv: 2005.09535 [cs.CR]. URL: https://arxiv.org/abs/2005.09535.

[OS23] Marc Ohm and Charlene Stuke. "Sok: Practical detection of software supply chain attacks". In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*. 2023, pp. 1–11.

[Pin+23] Donald Pinckney, Federico Cassano, Arjun Guha, and Jonathan Bell. *A Large Scale Analysis of Semantic Versioning in NPM*. 2023. arXiv: 2304.00394 [cs.SE]. URL: https://arxiv.org/abs/2304.00394.

[Pre] Tom Preston-Werner. *Semantic Versioning 2.0.0*. URL: %5Curl%7Bhttps://semver.org/%7D.

[Tea25a] Editorial Team. *npm registry attacked by secret-stealing worm*. 2025. URL: https://www.kaspersky.com/blog/tinycolor-shai-hulud-supply-chain-attack/54315/.

[Tea25b] HelixGuard Team. *Shai-Hulud Returns: Over 1K NPM Packages and 27K+ Github Repos infected via Fake Bun Runtime Within Hours*. 2025. URL: %5Curl%7Bhttps://helixguard.ai/blog/malicious-sha1hulud-2025-11-24/%7D.

[Wan+25] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. "MalPacDetector: An LLM-based Malicious NPM Package Detector". In: *IEEE Transactions on Information Forensics and Security* PP (Jan. 2025), pp. 1–1. DOI: 10.1109/TIFS.2025.3580336.

[Wei+25] Felix Weissberg, Lukas Pirch, Erik Imgrund, Jonas Möller, Thorsten Eisenhofer, and Konrad Rieck. "LLM-based Vulnerability Discovery through the Lens of Code Metrics". In: *arXiv preprint arXiv:2509.19117* (2025).

[Zah+25] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. *Leveraging Large Language Models to Detect npm Malicious Packages*. 2025. arXiv: 2403.12196 [cs.CR]. URL: https://arxiv.org/abs/2403.12196.

[Zha+25] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. "Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence". In: *ACM Transactions on Software Engineering and Methodology* 34.4 (Apr. 2025), pp. 1–28. ISSN: 1557-7392. DOI: 10.1145/3705304. URL: http://dx.doi.org/10.1145/3705304.

[Zor25] Marco Zoratti. "Valutazione di Metriche per il Rilevamento di Attacchi al Codice Sorgente". In: (2025). URL: https://unire.unige.it/handle/123456789/12801.